NAME : SAFIYA B

REG NO: 192411043

INDUSTRY BASED PROBLEM

main.c

Run

```c
#include <stdio.h>

#define STUDENTS 4
#define CONTENT 6

char *content[CONTENT] = {
    "C Programming",
    "Data Structures",
    "Algorithms",
    "DBMS",
    "Computer Networks",
    "Python"
};

/* Weighted Student-Content Graph
   0 = No interaction
   Higher value = Stronger interaction
*/
int graph[STUDENTS][CONTENT] = {
    {5, 4, 0, 2, 0, 3},
    {2, 5, 4, 0, 1, 0},
    {0, 3, 5, 4, 2, 0},
```

main.c

Run

```c
void recommendContent(int student)
{
    int score[CONTENT];
    int i, j;

    /* Calculate recommendation score */
    for (i = 0; i < CONTENT; i++)
    {
        score[i] = 0;

        /* Recommend only content not already studied */
        if (graph[student][i] == 0)
        {
            for (j = 0; j < CONTENT; j++)
            {
                score[i] += graph[student][j];
            }
        }
    }

    printf("\nRecommended Content:\n");
```

main.c

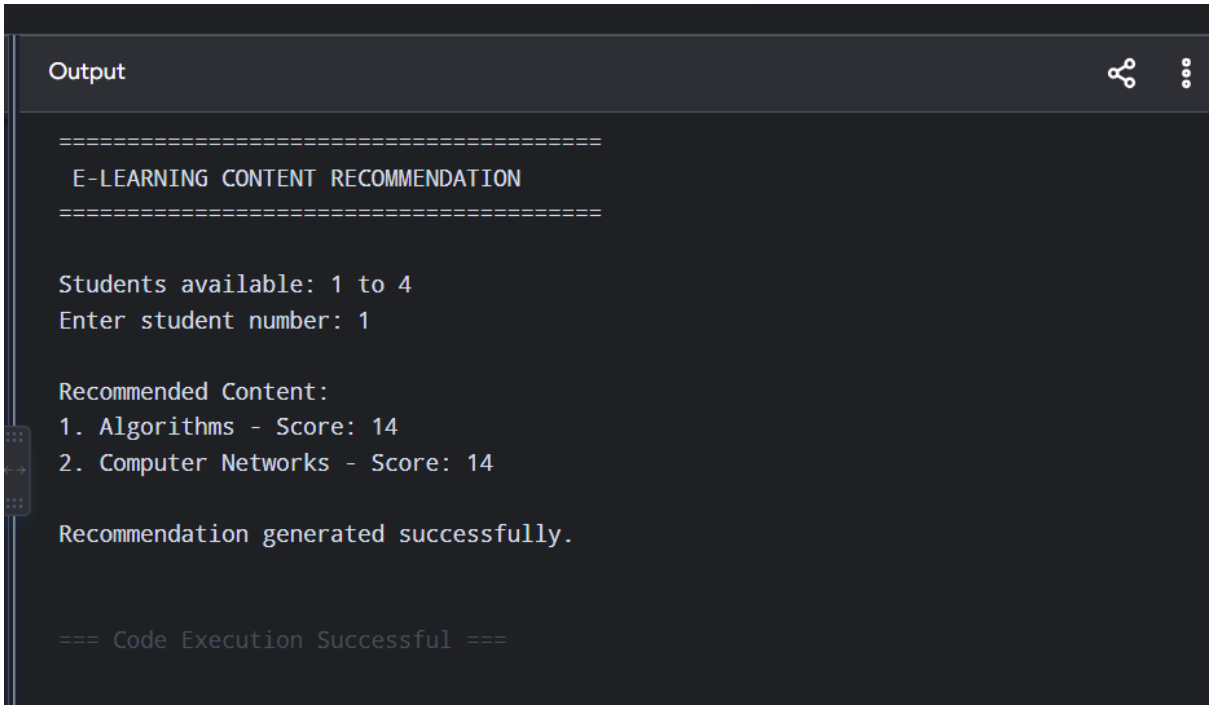
Run

```
28  {
51      {
52          int maxIndex = -1;
53          int maxScore = -1;
54
55          for (i = 0; i < CONTENT; i++)
56          {
57              if (score[i] > maxScore)
58              {
59                  maxScore = score[i];
60                  maxIndex = i;
61              }
62          }
63
64          if (maxIndex != -1 && maxScore > 0)
65          {
66              printf("%d. %s - Score: %d\n",
67                     rank, content[maxIndex], maxScore);
68
69              /* Prevent selecting the same content again */
70              score[maxIndex] = -1;
71          }
```

main.c

Run

```
76  {
78
79      printf("========================================\n");
80      printf(" E-LEARNING CONTENT RECOMMENDATION\n");
81      printf("========================================\n");
82
83      printf("\nStudents available: 1 to %d\n", STUDENTS);
84      printf("Enter student number: ");
85      scanf("%d", &student);
86
87      if (student < 1 || student > STUDENTS)
88      {
89          printf("Invalid student number.\n");
90          return 0;
91      }
92
93      recommendContent(student - 1);
94
95      printf("\nRecommendation generated successfully.\n");
96
97      return 0;
98  }
```



Implementation Explanation

1. A **weighted adjacency matrix** represents student-content interactions.
2. Each row represents a student.
3. Each column represents learning content.
4. The weight indicates the strength of the student's interaction.
5. Content already studied by the student is ignored.
6. A score is calculated for the remaining content.
7. The content is ranked based on its score.
8. The **Top-3 recommendations** are displayed.

Complexity

- **Time Complexity:** O(S × C)
- **Space Complexity:** O(S × C)

where S = number of students and C = number of learning contents.

This implementation demonstrates the **Graph + Ranking** approach required by Q33 and can be extended with PageRank/Personalized PageRank, sparse graphs, caching, and distributed processing for a large e-learning platform.